Agentic AI

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 18, 2026

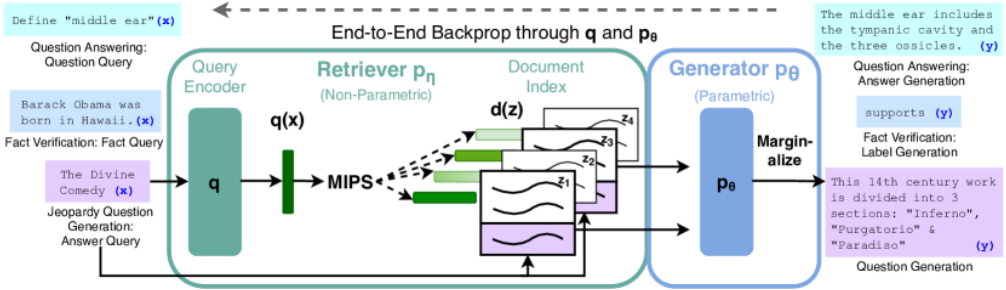
Retrieval-Augmented Generation (RAG)

Motivation for RAG

- LLMs are fixed after training: their knowledge cannot be easily updated.
- Updating facts or adding new information requires retraining or fine-tuning, which is costly and slow.
- Retrieval-Augmented Generation (RAG) addresses this by combining:
  - A **retriever** that fetches relevant documents or facts from an external knowledge base.
  - A **reader** (LLM) that generates answers conditioned on the retrieved information.
- This enables up-to-date, accurate, and context-aware responses without retraining the LLM.

Two main components:

- **Retriever:** Encodes the query and the documents into vectors and fetches the closest passages from the corpus by similarity search (e.g., DPR, ColBERT).
- **Generator** (also called the **reader**)**:** An LLM that produces the final answer conditioned on the retrieved passages (e.g., BART, GPT).



Source: Lewis et al., NeurIPS 2020, Fig. 1.

Example RAG Workflow

1. **User asks a question.**
2. **Retriever:** Embedding and similarity search retrieves top-k relevant documents from the knowledge base.
3. **Reader:** Retrieved documents and the user prompt are fed to the LLM.
4. **LLM generates a response** grounded in the retrieved documents.

RAG Tools and Libraries

- **LangChain:** Framework for building LLM-powered applications with retrieval, chaining, and orchestration.
- **LlamaIndex:** Data framework for connecting LLMs with external data sources and knowledge bases.
- **Haystack:** Open-source framework for building production-ready RAG pipelines.

Popular Vector Databases:

- **FAISS:** Facebook AI Similarity Search, efficient vector search library.
- **Weaviate:** Scalable, cloud-native vector database with RESTful APIs.
- **Qdrant:** Open-source vector search engine with filtering and payload support.

Benefits of RAG

- **Factual accuracy:** Answers are grounded in retrieved documents, reducing hallucinations.
- **Real-time updates:** Knowledge can be updated instantly by modifying the underlying corpus, without retraining the LLM.
- **Scalable with large corpora:** Efficient retrieval enables handling vast and dynamic knowledge bases.

RAG Limitations

- **Requires good retrieval quality:** Poor retrieval leads to irrelevant or incomplete context for the LLM.
- **Latency overhead:** Retrieval and reranking steps add to response time.
- **Domain-specific retrievers may require tuning:** Customization and fine-tuning are often needed for specialized domains.
- **Context length limits in LLMs:** Only a limited amount of retrieved content can be passed to the LLM at once.

Table of Contents

1. Motivation
2. Learning Outcomes
3. Agentic AI: Definitions, Reactive vs. Proactive Agents
4. Agent Loop, Tool Use, and Agentic RAG
5. Open-Source Agentic Frameworks
    1. A Taxonomy of Agent Frameworks
    2. Auto-GPT and BabyAGI
    3. Other Notable Frameworks

Table of Contents (contd.)

6. Code Generation
    1. Benchmarks and Metrics
    2. Methods: Context, Infilling, Retrieval, Execution Feedback
    3. Representative Code LMs
7. Continual Learning in Agents
8. Safety, Ethics, and Control
9. Future Directions

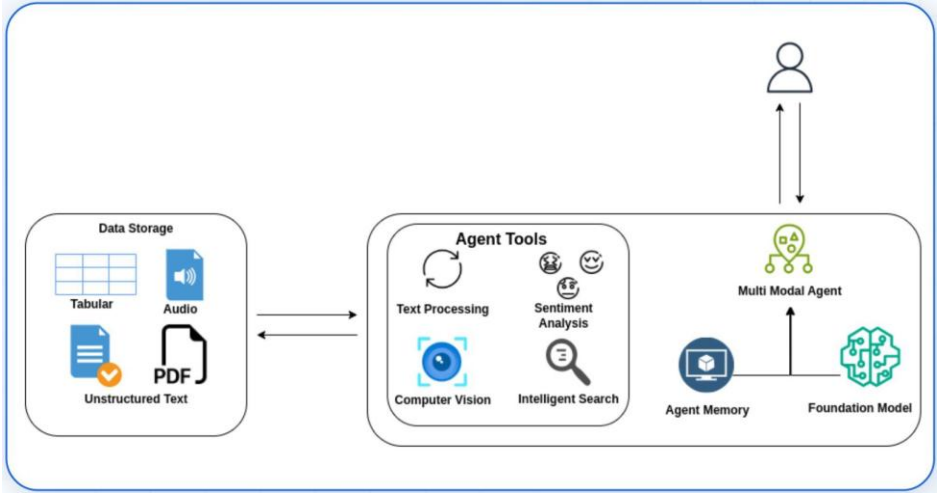
Motivation

- A language model answers; an **agent** acts: it pursues goals over multiple steps, using tools, memory, and feedback from its environment.
- Modern AI agents must perceive, reason, plan, and act in complex environments — often across multiple modalities, building on the vision-language models of the previous lecture.
- **Agentic AI** aims to build goal-directed, autonomous, and continuously learning systems on top of LLMs.
- The agent's most general interface to the world is **executable code** — so this lecture also covers how models write code, and how that ability is measured.
- The first agent frameworks (Auto-GPT, BabyAGI) opened new research frontiers in decision-making, robotics, and assistive AI — and new failure modes, which is why safety and control are part of the core material.

Learning Outcomes

- Understand the principles of agentic AI and agentic loops.
- Describe how an LLM acts through function calling, and what MCP standardises.
- Differentiate between reactive and proactive agent behavior.
- Explain how code generation is evaluated (HumanEval, pass@k, SWE-bench, match-based metrics) and the methods behind modern code LMs (context engineering, infilling, retrieval, execution feedback).
- Describe continual learning challenges in agentic systems.
- Critically evaluate frameworks like Auto-GPT and BabyAGI.
- Reflect on safety, control, and ethical issues in agentic AI.

Agentic AI


Data Storage
Tabular
Audio
PDF
Unstructured Text
Agent Tools
Text Processing
Sentiment Analysis
Computer Vision
Intelligent Search
Multi Modal Agent
Agent Memory
Foundation Model

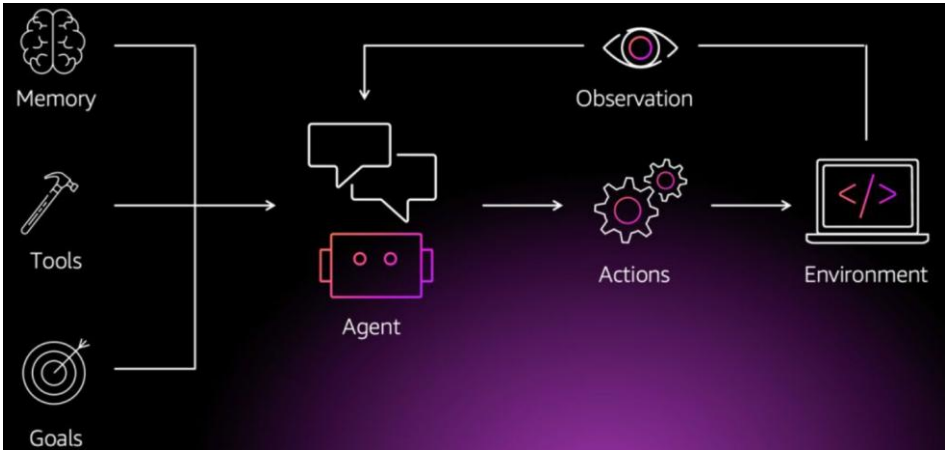
What is Agentic AI?

- AI systems that **perceive, reason, plan,** and **act** to fulfill goals.
- **Agent loop:** Observe → Plan → Act → Reflect → Learn
- Inspired by cognitive science, robotics, and autonomous agents.

Core Components:

- Memory
- Planning
- Tool-use (plugins, APIs)
- Feedback-driven behavior

What is Agentic AI?



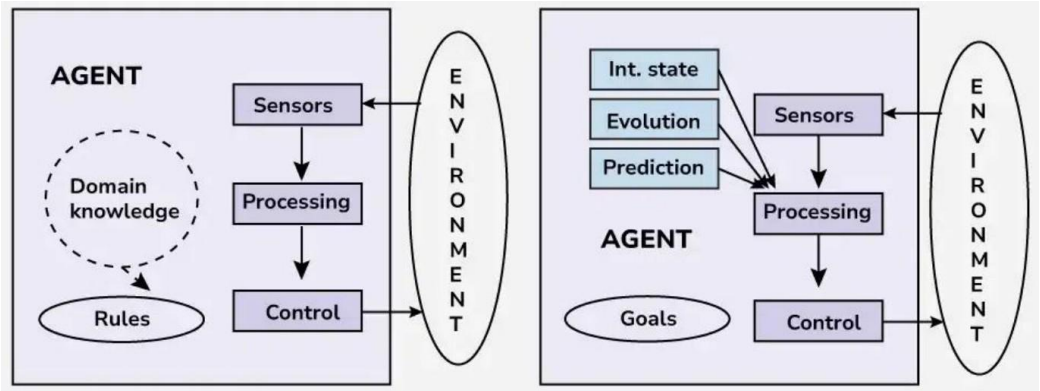
Source: Kipkemboi, Moura, Singh & Tsakpinis, “Build agentic systems with CrewAI and Amazon Bedrock”, AWS Machine Learning Blog, 2025.

Reactive vs. Proactive Agents

Feature	Reactive	Proactive
Response	Immediate	Goal-oriented
Planning	None	Yes
Learning	Event-triggered	Self-initiated
Example	Chatbots	Auto-GPT, Personal Assistants

Proactive agents generate goals, schedule actions, and evaluate results even without user prompts.

Reactive vs. Deliberative Agents



A reactive agent maps percepts to actions through fixed rules. A deliberative agent keeps internal state, predicts how the environment will evolve, and selects actions against a goal. Source: GeeksforGeeks, “Reactive vs Deliberative AI Agents”.

Agent Loop in Detail

▷ **Perceive:** Multimodal input (text, vision, memory)
▷ **Interpret:** NLP/vision models
▷ **Plan:** Task decomposition (e.g., LangChain chains)
▷ **Act:** API/tool calling
▷ **Reflect & Learn:** Refine memory or adjust goals

Applications:

- ▷Personal AI assistants
- ▷Research co-pilots
- ▷Automated workflows

Tool Use and Function Calling

- **Function calling** is how an LLM acts: each tool is described to the model as a function with a name, a description, and a JSON schema for its parameters.
- The model does not execute anything. It emits a structured call (function name + arguments); the runtime executes it and feeds the result back as a new message.
- **The loop**: model receives the task → emits a call → runtime returns the result → model reasons over it → another call or a final answer. This is the ReAct pattern covered earlier, with the action step now carried by the API instead of by parsed text.
- Modern APIs support **parallel calls** (several independent tools in one step) and **structured outputs** (forcing the reply itself into a schema).

Function Calling: Example

Tool definition given to the model:

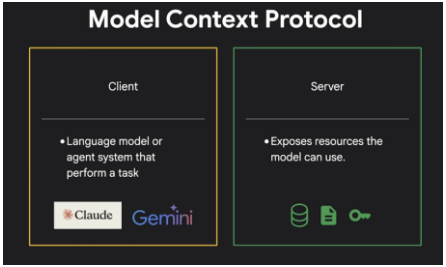
```
{"name": "get_weather",
 "description": "Current weather for a city",
 "parameters": {"type": "object",
   "properties": {"city": {"type": "string"}},
   "required": ["city"]}}
```

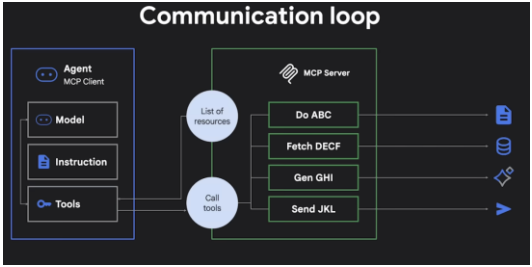
User: "Do I need an umbrella in Jeddah today?"

```
Model   -> call: get_weather(city="Jeddah")
Runtime-> result: {"condition": "rain", "temp_c": 24}
Model   -> "Yes, take one: it is raining in Jeddah (24 C)."
```

Standardising the Tool Interface: MCP

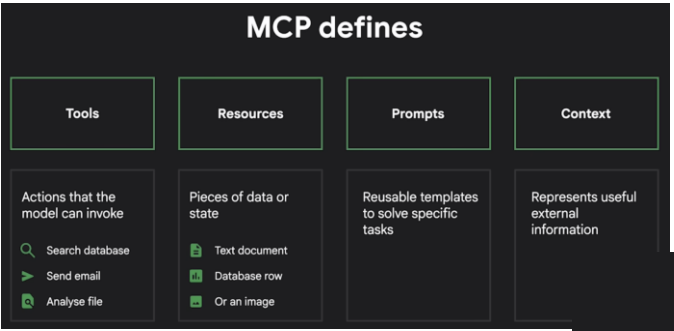
- **The problem.** Function calling describes tools *to one model, in one vendor's format*. Every new model or agent framework means rewriting the same integrations, so M agents times N tools costs $M \times N$ adapters.
- **Model Context Protocol (MCP),** released by Anthropic in November 2024, is an open protocol for that connection, modelled on the Language Server Protocol used by code editors. It reduces the cost to $M + N$.

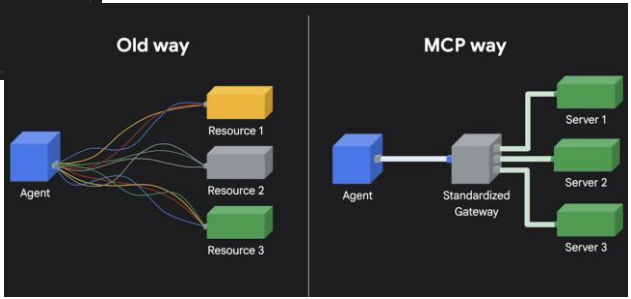




Specification: https://modelcontextprotocol.io/specification/latest https://github.com/google/mcp

Standardising the Tool Interface: MCP



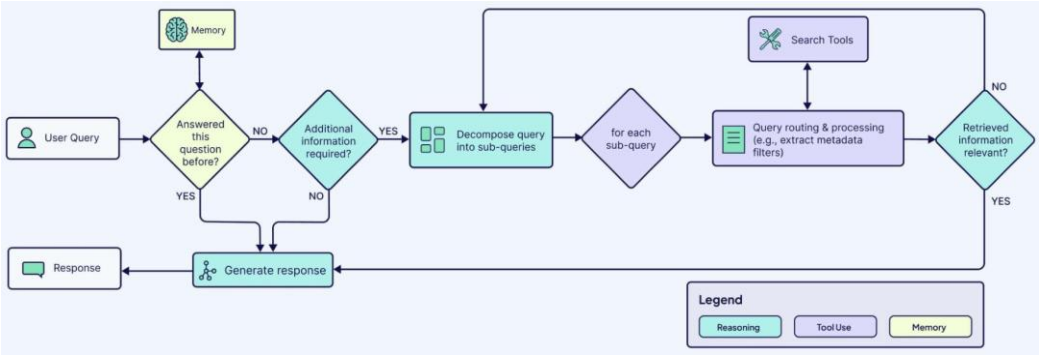


https://github.com/google/mcp

Agentic RAG

- **Fixed RAG pipeline:** retrieve once for the raw query, then generate. Fails when the query is vague, multi-part, or needs information the first retrieval missed.
- **Agentic RAG** makes retrieval a tool the model controls:
  - **Decide when:** answer directly from parametric knowledge, or retrieve.
  - **Rewrite the query:** decompose multi-part questions into targeted searches.
  - **Iterate:** retrieve, read, notice what is missing, retrieve again.
  - **Judge the evidence:** discard irrelevant passages before answering.
- Costs more latency and tokens per query; pays off on multi-hop questions and heterogeneous document collections.

Agentic RAG Workflow



Source: Newhauser, Yadav, Monigatti & Çelik, “What Are Agentic Workflows? Patterns, Memory, Use Cases, and Examples”, Weaviate blog, 2025.

Open-Source Agentic Frameworks

Auto-GPT

- **Open-source project** (Toran Bruce Richards, March 2023) that chained GPT-4 calls into a self-directed agent.
- Used **long-term memory** in a vector store, a file workspace, and self-generated tasks.
- **Requirements:**
  - Task input
  - Internet / API access
  - Feedback loop
- **Status:** the original agent now ships as `classic/` and is unsupported; the project has become a visual builder, the AutoGPT Platform.

GitHub: https://github.com/Significant-Gravitas/AutoGPT

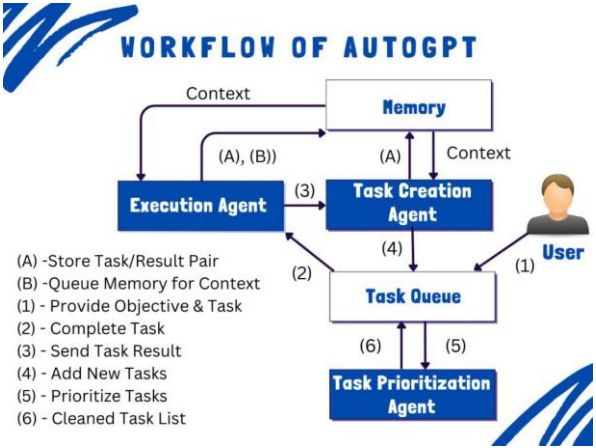
BabyAGI

- **Lightweight Python agent loop** (Yohei Nakajima, March 2023), around 140 lines
- Generates tasks from a high-level objective
- Runs tasks, stores results in a vector store, and generates new tasks iteratively
- Emphasizes simplicity and minimalism

Original code: https://github.com/yoheinakajima/babyagi_archive

The babyagi repository itself now hosts a different, self-building agent framework.

BabyAGI: The Task-Driven Loop



Three LLM calls with one shared task queue and one memory store. Architecture after Y. Nakajima, “Task-Driven Autonomous Agent” (2023); this rendition is labelled “Workflow of AutoGPT” by its publisher, but the three named agents are BabyAGI’s.

BabyAGI: How BabyAGI Works

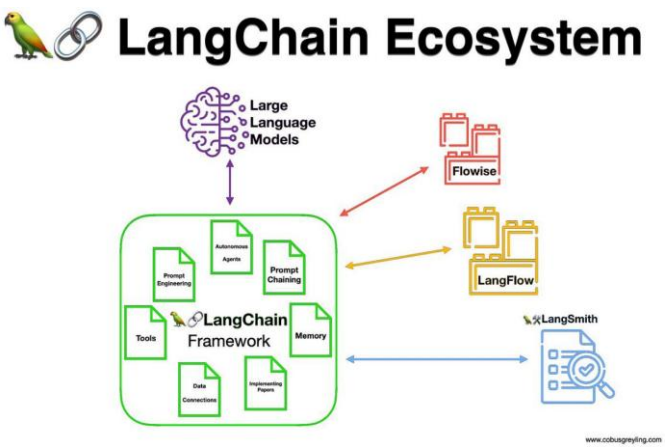
What the First Wave Taught Us

- **The measured results were poor.** GAIA asks 466 questions that are conceptually simple for a person but need tool use to answer. Human respondents score 92%; GPT-4 with plugins scores 15%. Auto-GPT on a GPT-4 backend manages 14.4% on the easiest tier and 0.4% on the middle tier, behind plain GPT-4 with hand-picked plugins.
- **Where the loops broke:** no reliable stopping criterion, so agents revisited the same subtask; errors compounded because nothing checked a step before the next built on it; and cost grew with every retry.
- AgentBench, across eight interactive environments, traces the gap to long-term reasoning, decision-making, and instruction-following rather than to the scaffolding around the model.
- **What survived:** the loop and its parts, a task queue, a memory store, tools behind a schema. What did not is the assumption that full autonomy from a one-line objective is the right target. Current systems narrow the task, verify each step, and keep a person in the loop for consequential actions.

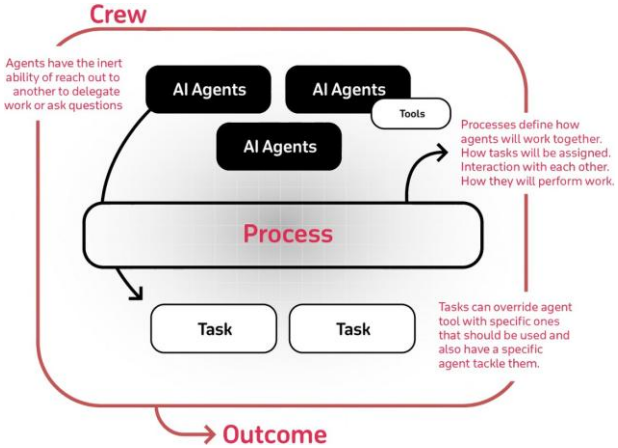
Sources: Mialon et al., “GAIA”, arXiv:2311.12983, Table 4; Liu et al., “AgentBench”, ICLR 2024, arXiv:2308.03688.

Other Notable Frameworks

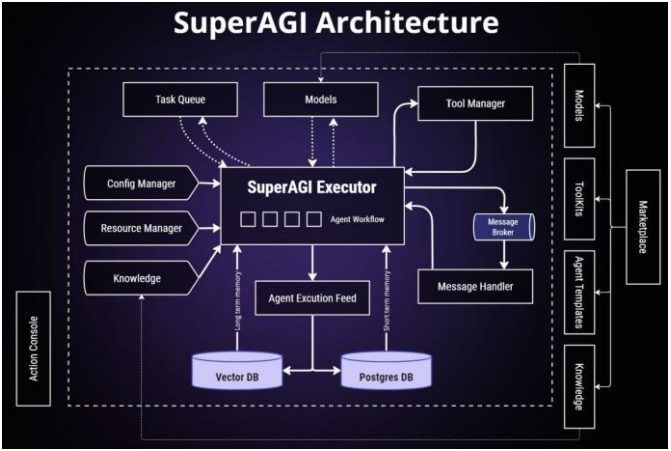
- **LangChain:** Framework for chaining LLMs with tools, APIs, and memory; since version 1.0 (2025) its headline abstraction is agent construction on the LangGraph runtime.
- **CrewAI:** Python framework and hosted platform for orchestrating role-playing agents that collaborate on a task.
- **SuperAGI:** Dev-first open-source autonomous-agent framework with toolkits, agent memory, and telemetry. Largely dormant: last release January 2024.
- **OpenHands** (renamed from OpenDevin in August 2024): open platform for software-development agents, giving the agent a sandboxed shell, editor, and browser behind an event-stream architecture.



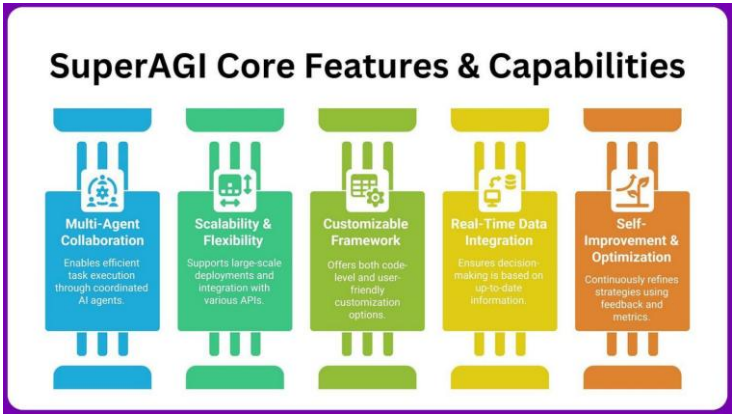
Source: C. Greyling, “The Growing LangChain Ecosystem”.



A crew is a set of role-playing agents; a process defines how they hand work between each other. Source: CrewAI documentation, https://docs.crewai.com.

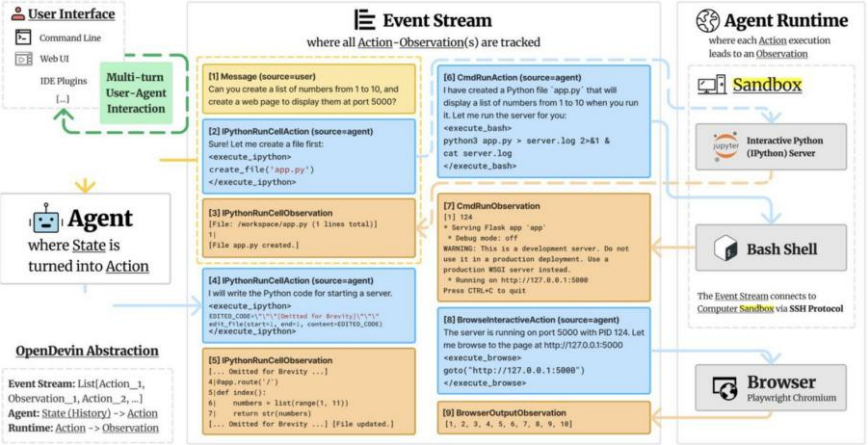


Source: SuperAGI project, https://github.com/TransformerOptimus/SuperAGI.



Source: Servifyspheresolutions, “AutoGPT vs. BabyAGI vs. SuperAGI: Which AI Agent is Smarter for Autonomous Tasks?”, Generative AI (Medium), 2025.

OpenHands



Every action and observation is appended to one event stream, which doubles as the agent's history and its memory. Source: Wang et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents", ICLR 2025, arXiv:2407.16741.

Code Generation

- Generate executable code as an interface to a program
- Why do it?
  - Accelerates software engineering
  - Enables models to access tools (more in a later class)

Inputs and Outputs

Natural language

Code context

Inputs and outputs

Screenshots

Code

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

    def years_until_100(self):
        years_left = 100 - self.age
        if years_left <= 0:
            print(f"{self.name} is already 100 years old or more!")
        else:
            print(f"{self.name} has {years_left} years until they turn 100.")
```

Example: Github Copilot

- A code completion engine within your IDE (VSCode)

```
# load hello.csv
data = np.loadtxt('hello.csv', delimiter=',', skiprows=1)

# add together the two largest numbers in the first column
result = np.sum(np.sort(data[:, 0])[-2:])

print how much bigger these two numbers are than the average of the first column
result = result - np.mean(data[:, 0])
```

https://youtu.be/xe3ol0rTeaQ

Example: Claude 3

G Create a React app that looks like this screenshot.

To create a React app that looks similar to the screenshot, you can follow these steps:

1. Set up a new React project using Create React App or your preferred method.
2. Create a component for the chat interface, such as `ChatInterface`.
3. Inside the `ChatInterface` component, add a header with the app name "Claude"

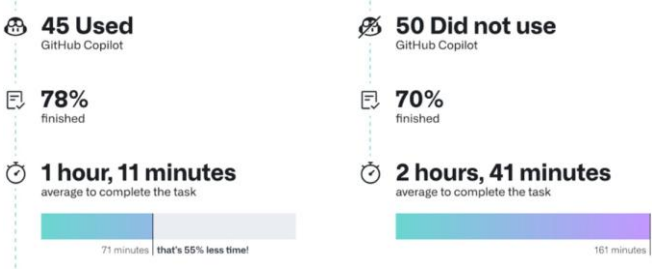
Here's a simplified example of the `ChatInterface` component:

jsx Copy code

```jsx
import React from 'react';
import styled from 'styled-components';

const ChatContainer = styled.div`
  /* Add styles for the chat container */
`;

const Header = styled.header`
  /* Add styles for the header */
`;

const ChatSection = styled.section`
  /* Add styles for the chat section */
`;

const CapabilityCard = styled.div`
  /* Add styles for the capability cards */
`;

const ChatInterface = () => {
  return (
    <ChatContainer>
      <Header>
```

Why do we do it?

- Making software is important!
  - US software developers make $175B per year
- Code generation leads to large improvements in productivity (Github 2023)



Differences Between code and Natural Language

- Code has strict grammar
- We know the code semantic flow
- Code is executable
- Code is created more incrementally

Code Generation - Subtasks and Datasets

HumanEval (chen et al. 2021)

- Examples of usage of the Python standard library
- 164 test examples
- Includes docstring, some example inputs/outputs, and tests

```python
def solution(lst):
    """Given a non-empty list of integers, return the sum of all of the odd elements
    that are in even positions.

    Examples
    solution([5, 8, 7, 1]) ==>12
    solution([3, 3, 3, 3, 3]) ==>9
    solution([30, 13, 24, 321]) ==>0
    """
    return sum(lst[i] for i in range(0,len(lst)) if i % 2 == 0 and lst[i] % 2 == 1)
```

```python
def encode_cyclic(s: str):
    """
    returns encoded string by cycling groups of three characters.
    """
    # split string to groups. Each of length 3.
    groups = [s[(3 * i):min((3 * i + 3), len(s))] for i in range((len(s) + 2) // 3)]
    # cycle elements in each group. Unless group has fewer elements than 3.
    groups = [(group[1:] + group[0]) if len(group) == 3 else group for group in groups]
    return "".join(groups)

def decode_cyclic(s: str):
    """
    takes as input string encoded with encode_cyclic function. Returns decoded string.
    """
    # split string to groups. Each of length 3.
    groups = [s[(3 * i):min((3 * i + 3), len(s))] for i in range((len(s) + 2) // 3)]
    # cycle elements in each group.
    groups = [(group[-1] + group[:-1]) if len(group) == 3 else group for group in groups]
    return "".join(groups)
```

Metric: Pass@K (Chen et al. 2021)

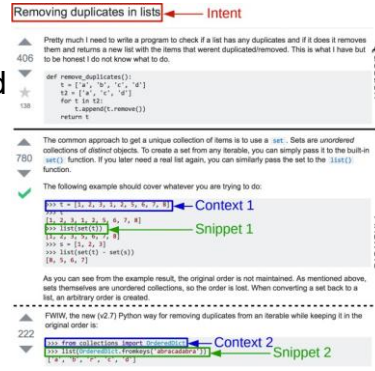
- Basic idea: “if we generate *K* examples, will at least one of them pass unit tests”
- Generating only *K* will result in high variance, so we generate *N* > *K* with *C* correct answers, and then calculate expected value

$$\text{pass@}k := \mathop{\mathbb{E}}_{\text{Problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

Broader Domains: CoNaLa/ODEX

(Yin et al. 2018, Wang et al. 2022)

- CoNaLa: Broader data scraped from StackOverflow
- ODEX: Adds execution-based evaluation
- Wider variety of libraries



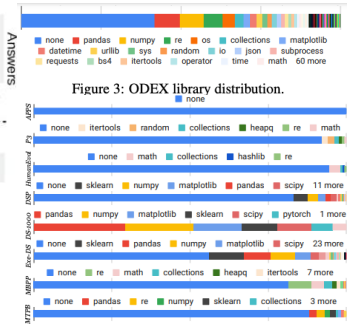
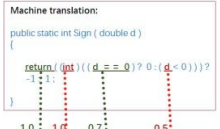


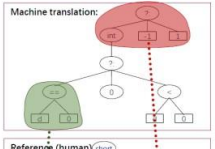
Figure 3: ODEX library distribution.

Metric: BLEU, CodeBLEU

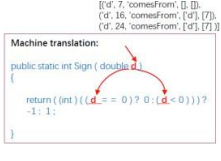
- Issues w/ execution-based evaluation:
  - Requires that code be easily executable (requires unit tests and hard in large libraries)
  - Ignores stylistic considerations
- **BLEU:** consider text n-gram overlap with human code
- **CodeBLEU:** also considers syntax and semantic flow (Ren et al. 2020)



Weighted N-Gram Match



Syntactic AST Match



Semantic Data-flow Match

Metric: CodeBERTScore

- It is also possible to use embedding-based methods to compare code
- **CodeBERTScore:** BERTScore with CodeBERT trained on lots of code (Zhou et al. 2023)
- Leads to better correlation w/ human judgement and execution accuracy

Metric	Java		C++		Python		JavaScript	
	τ	r_s	τ	r_s	τ	r_s	τ	r_s
BLEU	.481	.361	.112	.301	.393	.352	.248	.343
CodeBLEU	.496	.324	.175	.201	.366	.326	.261	.299
ROUGE-1	.516	.318	.262	.260	.368	.334	.279	.280
ROUGE-2	.525	.315	.270	.273	.365	.322	.261	.292
ROUGE-L	.508	.344	.258	.288	.338	.350	.271	.293
METEOR	**.558**	**.383**	.301	.321	.418	.402	**.324**	**.415**
chrF	.532	.319	.319	.321	.394	.379	.302	.374
CrystalBLEU	.471	.273	.046	.095	.391	.309	.118	.059
CodeBERTScore	**.553**	.369	**.327**	**.393**	**.422**	**.415**	**.319**	.402

Table 1: Kendall-Tau (τ) and Spearman (r_s) correlations of each metric with the functional correctness on HumanEval in multiple languages. The correlation coefficients are reported as the average across three runs. Standard deviation is provided in Table 3.

Metric	τ	r_p	r_s
BLEU	.374	.604	.543
CodeBLEU	.350	.539	.495
ROUGE-1	.397	.604	.570
ROUGE-2	.429	.629	.588
ROUGE-L	.420	.619	.574
METEOR	.366	.581	.540
chrF	.470	.635	.623
CrystalBLEU	.411	.598	.576
CodeBertScore	**.517**	**.674**	**.662**

Table 2: The Kendall-Tau (τ), Pearson (r_p) and Spearman (r_s) correlation with human preference. The best performance is **bold**. The correlation coefficients are reported as the average across three runs. Standard deviations are provided in Table 4.

Data Science Notebooks: ARCADE

(Yin et al. 2022)

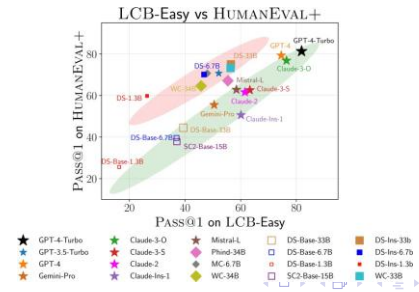
- Data science notebooks (e.g. Jupyter) allow for incremental implementation
- Allows evaluation of code in context

Figure 1: An example of a computational notebook adapted from our dataset, with examples of reading and preprocessing data (cell c_1), data wrangling (cell c_2, c_3), and data analysis (cells $c_3 - c_7$). Annotated NL intents are shown in green.

An Aside: Dataset Leakage

- Leakage of datasets is a big problem
- ARCADE shows that novel notebooks are harder than online notebooks
- LiveCodeBench (Jain et al. 2023) shows that some code LMs outperform on HumanEval

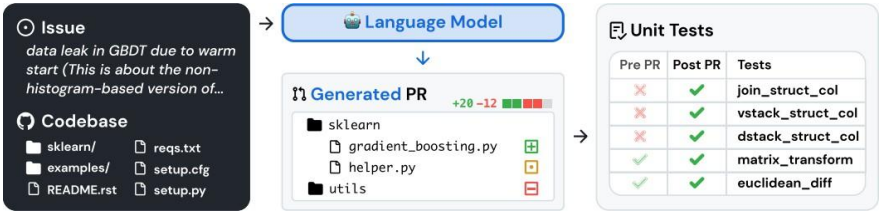
pass@k	Existing 5	New 5
InCoder 1B	30.1	3.8
InCoder 6B	41.3	7.0
CodeGen$_{multi}$ 350M	13.3	1.0
CodeGen$_{multi}$ 2B	25.0	2.7
CodeGen$_{multi}$ 6B	28.0	3.0
CodeGen$_{multi}$ 16B	31.2	4.6
CodeGen$_{mono}$ 350M	18.9	1.9
CodeGen$_{mono}$ 2B	35.8	6.5
CodeGen$_{mono}$ 6B	42.1	8.9
CodeGen$_{mono}$ 16B	46.7	12.0
PaLM 62B (1.3T Tokens)	49.7	12.5
+ Python Code	58.8 +9.1	21.4 +8.9
+ Notebooks (PaChiNCo)	64.6 +7.8	30.6 +9.2
− Schema Description	60.5 −4.1	22.7 −7.9



Dataset: SWEBench

(Jiminez et al. 2023)

- Issues from GitHub + codebases -> pull request

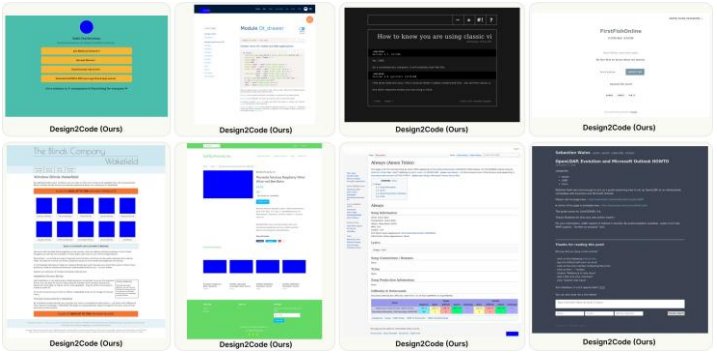


- Requires long-context understanding, precise implementation

Dataset: Design2Code

(Si et al. 2024)

- Code generation from web sites



- Also proposed Design2Code model

Code Generation - Methods

Basic Method: Code-generating LM

- Feed the previous code to an LM
- Virtually all serious LMs are trained on code nowadays, and have the ability to complete queries
- More on specific LMs later!
- Note: temperature settings are important, set to lower values

Code Infilling

(Fried et al. 2022)

- In code generation, we often want to fill in code
- Solution: train for infilling

Training

Original Document

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        word_counts = {}
        for line in f:
            for word in line.split():
                if word in word_counts:
                    word_counts[word] += 1
                else:
                    word_counts[word] = 1
    return word_counts
```

Masked Document

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        <MASK:0> in word_counts:
                word_counts[word] += 1
            else:
                word_counts[word] = 1
    return word_counts
<MASK:0> word_counts = {}
        for line in f:
            for word in line.split():
                if word <EOM>
```

Lots of Available Information for Coding!

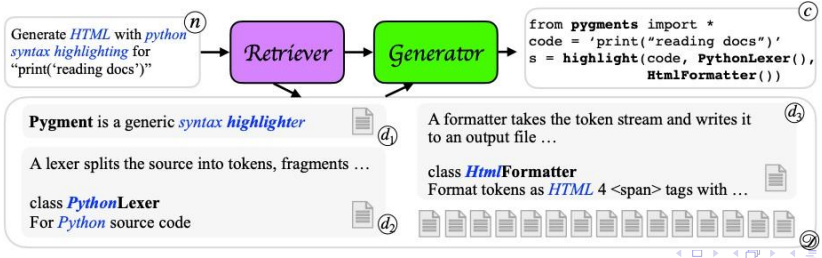
- Current code context
- Description of issue to fix
- Repo context
- Open tabs

Example: Copilot Prompting Strategy (Thakkar 2023)

- **Extract prompt** given current doc and cursor position
- Identify **relative path and language**
- Find **most recently accessed** 20 files of the same language
- **Include:** text before, text after, similar files, imported files, metadata about language and path
- **TL;DR:** lots of prompt engineering to get most useful context in the prompt

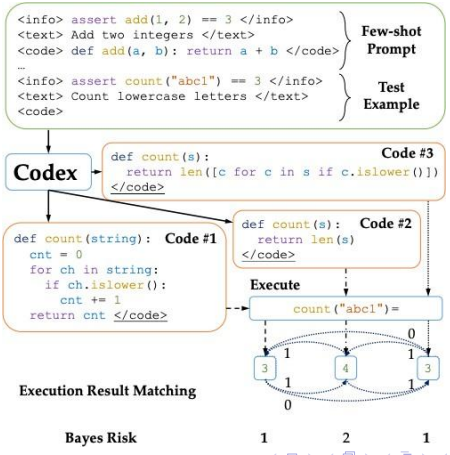
Retrieval-based Code Generation

- Retrieve similar code from online, and fill it in with a retrieval-augmented LM (Hayati et al. 2018)
- Particularly, in code there is also documentation, which can be retrieved (Zhou et al. 2022)



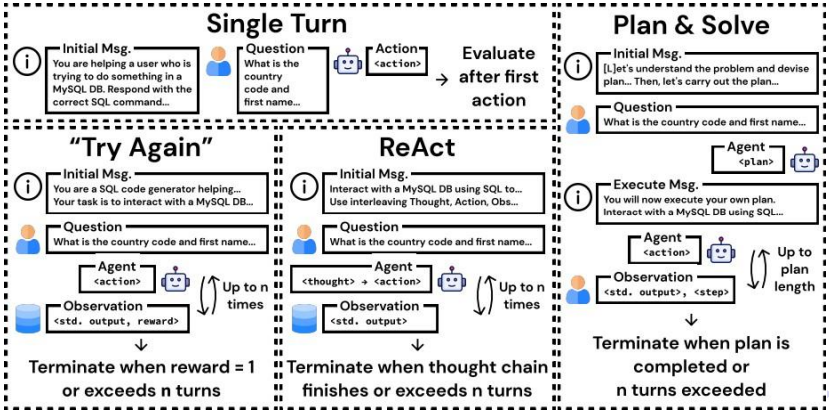
Execution Feedback (Shi et al. 2022)

- Code can be executed, so we can use this to check results!



Fixing Based on Error Messages

- e.g. InterCode (Yang et al. 2023)



Future Directions

Open Problems and Research Directions

- **Multimodal grounding in real-world environments:** How can agents connect language, vision, and action in dynamic, noisy settings? SayCan is one line of attack: score what the language model *wants* to do against what the robot can actually do here.
- **World model integration:** Give the agent a learned internal model of how its environment evolves, so it can plan against predicted states rather than only react to observed ones.
- **Agent collaboration:** Enabling effective communication and coordination in multi-agent systems.
- **Scaling memory-efficient agents:** Designing architectures that scale to long-term, large-scale memory without prohibitive costs.
- **Robust reward models:** Developing reliable reward and feedback mechanisms for open-ended, real-world tasks.
- **Meta-cognition:** Building agents that can introspect, self-monitor, and adapt their own reasoning processes.

Summary

- **Agentic AI** adds goals, memory, and tools to a language model. **Function calling** is the mechanism; **MCP** standardises the interface.
- **The first autonomous frameworks (Auto-GPT, BabyAGI)** established the loop but performed poorly on real tasks. Their components survived; the assumption of full autonomy did not.
- **Code generation** is the agent's strongest actuator: executability gives cheap, trustworthy feedback — unit-test evaluation (pass@k, SWE-bench), execution-driven repair, and verifiable rewards all flow from it.
- **Continual learning** remains open: agents must absorb new tasks without overwriting old ones.
- Agency turns a wrong answer into a wrong action, which is why safety and oversight get their own treatment next.

References

1 Yao, S., Zhao, J., Yu, D., et al. (2023).
ReAct: Synergizing Reasoning and Acting in Language Models.
ICLR 2023. arXiv:2210.03629

2 Anthropic (2024).
Model Context Protocol: specification and SDKs.
https://modelcontextprotocol.io/specification/latest

3 Richards, T. B. (Torantulino) et al. (2023).
AutoGPT.
https://github.com/Significant-Gravitas/AutoGPT

4 Nakajima, Y. (2023).
BabyAGI (original task-driven agent; archived 2024).
https://github.com/yoheinakajima/babyagi_archive

5 Wang, X., Li, B., Song, Y., et al. (2025).
OpenHands: An Open Platform for AI Software Developers as Generalist Agents.
ICLR 2025. arXiv:2407.16741

References (cont.)

6 Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., Scialom, T. (2023).
GAIA: a Benchmark for General AI Assistants.
arXiv:2311.12983

7 Liu, X., Yu, H., Zhang, H., et al. (2024).
AgentBench: Evaluating LLMs as Agents.
ICLR 2024. arXiv:2308.03688

8 Ahn, M., Brohan, A., Brown, N., Chebotar, Y., et al. (2022).
Do As I Can, Not As I Say: Grounding Language in Robotic Affordances.
CoRL 2022, PMLR 205:287–318. arXiv:2204.01691

9 Xie, J., Chen, Z., Zhang, R., et al. (2024).
Large Multimodal Agents: A Survey.
arXiv:2402.15116

10 Chen, M., Tworek, J., Jun, H., et al. (2021).
Evaluating Large Language Models Trained on Code (Codex, HumanEval, pass@k).
arXiv:2107.03374

References (cont.)

11 Jimenez, C., Yang, J., Wettig, A., et al. (2024).
SWE-bench: Can Language Models Resolve Real-World GitHub Issues?
ICLR 2024. arXiv:2310.06770

12 Jain, N., Han, K., Gu, A., et al. (2024).
LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.
arXiv:2403.07974

13 Ren, S., Guo, D., Lu, S., et al. (2020).
CodeBLEU: a Method for Automatic Evaluation of Code Synthesis.
arXiv:2009.10297

14 Zhou, S., Alon, U., Agarwal, S., Neubig, G. (2023).
CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code.
EMNLP 2023. arXiv:2302.05527

15 Fried, D., Aghajanyan, A., Lin, J., et al. (2022).
InCoder: A Generative Model for Code Infilling and Synthesis.
ICLR 2023. arXiv:2204.05999

References (cont.)

16 Zhou, S., Alon, U., Xu, F., Wang, Z., Jiang, Z., Neubig, G. (2022).
DocPrompting: Generating Code by Retrieving the Docs.
ICLR 2023. arXiv:2207.05987

17 Yang, J., Prabhakar, A., Narasimhan, K., Yao, S. (2023).
InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback.
NeurIPS 2023. arXiv:2306.14898

18 Rozi`ere, B., Gehring, J., Gloeckle, F., et al. (2023).
Code Llama: Open Foundation Models for Code.
arXiv:2308.12950

19 Lozhkov, A., Li, R., Allal, L. B., et al. (2024).
StarCoder 2 and The Stack v2: The Next Generation.
arXiv:2402.19173

20 Guo, D., Zhu, Q., Yang, D., et al. (2024).
DeepSeek-Coder: When the Large Language Model Meets Programming.
arXiv:2401.14196

References (cont.)

21 Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M. (2023).
The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.
arXiv:2302.06590

22 Thakkar, P. (2023).
Copilot Internals (reverse engineering the Copilot prompt).
thakkarparth007.github.io/copilot-explorer